

Endpoints SUPABASE

id REQUIRED Type number Format bigint Description Click to edit.	Select id <pre>let { data: tasks, error } = await supabase .from('tasks') .select('id')</pre>
title REQUIRED Type string Format text Description Click to edit.	Select title <pre>let { data: tasks, error } = await supabase .from('tasks') .select('title')</pre>
subject REQUIRED Type string Format text Description Click to edit.	Select subject <pre>let { data: tasks, error } = await supabase .from('tasks') .select('subject')</pre>
description OPTIONAL Type string Format text Description Click to edit.	Select description <pre>let { data: tasks, error } = await supabase .from('tasks') .select('description')</pre>
due_date OPTIONAL Type string Format text Description Click to edit.	Select due_date <pre>let { data: tasks, error } = await supabase .from('tasks') .select('due_date')</pre>
priority OPTIONAL Type string Format text Description Click to edit.	Select priority <pre>let { data: tasks, error } = await supabase .from('tasks') .select('priority')</pre>
status OPTIONAL Type string Format text Description Click to edit.	Select status <pre>let { data: tasks, error } = await supabase .from('tasks') .select('status')</pre>
user_email REQUIRED Type string Format text Description Click to edit.	Select user_email <pre>let { data: tasks, error } = await supabase .from('tasks') .select('user_email')</pre>

task_type OPTIONAL Type string Format text Description Click to edit.	Select task_type <pre>let { data: tasks, error } = await supabase .from('tasks') .select('task_type')</pre>
created_at REQUIRED Type string Format timestamp with time zone Description Click to edit.	Select created_at <pre>let { data: tasks, error } = await supabase .from('tasks') .select('created_at')</pre>

Read rows To read rows in <code>tasks</code> , use the <code>select</code> method. Learn more Filtering Supabase provides a wide range of filters. Learn more	Read all rows <pre>let { data: tasks, error } = await supabase .from('tasks') .select('*')</pre> Read specific columns <pre>let { data: tasks, error } = await supabase .from('tasks') .select('some_column,other_column')</pre> Read referenced tables <pre>let { data: tasks, error } = await supabase .from('tasks') .select(`     some_column,     other_table (       foreign_key     )   `)</pre> With pagination <pre>let { data: tasks, error } = await supabase .from('tasks') .select('*') .range(0, 9)</pre> With filtering <pre>let { data: tasks, error } = await supabase .from('tasks') .select(".*")</pre>
---	--

Insert rows

`insert` lets you insert into your tables. You can also insert in bulk and do UPSERT.

`insert` will also return the replaced values for UPSERT.

[Learn more](#)

Insert a row

```
const { data, error } = await supabase
  .from('tasks')
  .insert([
    { some_column: 'someValue', other_column: 'otherValue' },
  ])
  .select()
```

Insert many rows

```
const { data, error } = await supabase
  .from('tasks')
  .insert([
    { some_column: 'someValue' },
    { some_column: 'otherValue' },
  ])
  .select()
```

Upsert matching rows

```
const { data, error } = await supabase
  .from('tasks')
  .upsert({ some_column: 'someValue' })
  .select()
```

Update rows

`update` lets you update rows. `update` will match all rows by default. You can update specific rows using horizontal filters, e.g. `eq`, `lt`, and `is`.

`update` will also return the replaced values for UPDATE.

[Learn more](#)

Update matching rows

```
const { data, error } = await supabase
  .from('tasks')
  .update({ other_column: 'otherValue' })
  .eq('some_column', 'someValue')
  .select()
```

Delete rows

`delete` lets you delete rows. `delete` will match all rows by default, so remember to specify your filters!

[Learn more](#)

Delete matching rows

```
const { error } = await supabase
  .from('tasks')
  .delete()
  .eq('some_column', 'someValue')
```